P4-NEON: Multihoming in Software Defined Network VANETs with P4

David Caetano, Tiago Santos, Bruno Sousa
University of Coimbra, CISUC/LASI,
Department of Informatics Engineering, Coimbra, Portugal
dcaetano@student.dei.uc.pt, tiagoms@student.dei.uc.pt, bmsousa@dei.uc.pt

2025 IEEE 50th Conference on Local Computer Networks (LCN) | 979-8-3315-3703-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/LCN65610.2025.11146327

***Abstract*—Providing Quality of Service (QoS) in Vehicular ad hoc networks (VANETs) is challenging, due to the numerous connections that vehicles can maintain, termed multihoming setups. P4-NEON employs a multi-criteria decision-making algorithm to improve multihoming in VANETs by leveraging segment routing within the IPv6 data plane. The entire data plane is programmed with P4 and uses In-band Network Telemetry to collect relevant performance data of ongoing data flows. This information improves Quality of Service decisions, such as minimizing delay and packet loss. P4-NEON is an all-encompassing solution featuring visualization tools for presenting live network statistics and active data flow paths. By employing segment routing within the IPv6 data plane, P4-NEON achieved better load balancing in the scenario studied, effectively lowered the average delay for Emergency Flows relative to others, and decreased the occurrence of out-of-order packets, all while maintaining other performance metrics evaluated during testing.**

***Index Terms*—VANET, SDN, Multihoming, INT, QoS, Segment Routing, SRv6**

I. Introduction

Vehicular Ad Hoc Networks (VANET) is described as a key strategy to improve collision prevention, road hazards alerts, and the transmission of safety-related messages [1]. However, managing these networks has challenges, particularly considering multiple access technologies, known as multihoming.

The Software Defined Networks (SDN) paradigm allows for easy management and scaling of resources, but is not adapted to the characteristics of VANET, specifically mobility management and monitoring of dynamic networks. Programming Protocol-independent Packet Processors (P4) is an approach that allows the programming of the behavior of the data plane more efficiently and directly than conventional SDN technologies such as OpenFlow (OF). A C-like language that allows for full programmability of pipelines, actions, parsing, and deparsing of packets, at near line-rate packet processing speeds [2]. Changes can be made at any moment in a centralized manner via a SDN controller with no need to change or update the hardware, while still supporting optional external functionalities that the manufacturer may still choose to implement, these features make it very useful for the challenges presented by VANET. Multihoming is a technique for traffic management that allows for the simultaneous usage of multiple equal cost paths, between nodes of the network, in a scenario like VANET, where the topology is always changing and where the density of the nodes is very high, the possibility of using all these wireless paths between vehicles can result in a performance gain.

Nuno Ferreira et al. [3] already consider the usage of SDN for multihomed VANET. However, the implemented solution does not address the issue of efficient and scalable statistical data collection to support decision-making. Additionally, it depends on OF, which, due to its protocol and target-specific nature, limits the experimental versatility provided by P4. The monitoring of data flows in SDN is complex, specifically in dynamic environments such as VANETs. The In-band Network Telemetry (INT) framework [4] is a solution that allows the collection of statistical data from data flows, with low overhead [5], and facilitates its deployment in VANETs, being compatible with technologies such as P4 [5] and OF [6].

The main goal of P4-NEON is to automate the load balancing support in multihomed VANETs through a programmable data plane in P4 and with support for Internet Protocol version 6 (IPv6) and IPv6 Segment Routing (SRv6), which uses micro Segment ID (uSID). Monitoring data is collected using the INT framework, which increases the interoperability of the project with other tools that also use INT data to work, such as the project [7]. Although the context of this project is VANET, P4-NEON can be applied in any SDN.

P4-NEON employs a Multiple-criteria decision analysis (MCDA) to improve multihoming in VANETs by using SRv6, allowing improved Quality of Service (QoS) decisions. In the scenarios evaluated, compared to base Equal-cost multi-path routing (ECMP), P4-NEON was able to reduce the number of out-of-order packets by 43.41%, reduce the Standard Deviation (STD) of the % of packets per switch by -0.33%, without changing its Average (AVG), and improve the flow latency of the emergency flow compared to the other ones by 83.98%.

The remainder of the manuscript is organized as follows: Section II presents the background, Section III describes the P4-NEON solution. Section IV details the implementations and evaluation methodology, while Section V presents and analyzes the results achieved. Section VI concludes the paper.

II. Related Work

Open-source implementations [8] enable SRv6 in SDN, where the switches use P4, allowing to establish more than one node to which the packet must travel, using unidirectional Anycast or Nodes instructions.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:43:26 UTC from IEEE Xplore. Restrictions apply.

Kuntz et al. [9] consider IPv6 to improve multihoming support in mobile networks, but does not consider VANET and/or emergency applications.

Nuno Ferreira et al. [3] implement a real-life tested scenario for the usage of Multihoming for SDN VANET. The vehicles were equipped with On Board Unit (OBU) that enabled several wireless technologies (ITS-G5 and cellular), for communication with both Road Side Unit (RSU) and cellular stations. The devices used OF, OpenvSwitch (OVS) for wireless interfaces, and Ryu as the SDN Controller, which is not scalable.

Singh et al. [5] implemented and tested the usage of P4 and INT on a SDN while using real hardware in links with a data rate of 40 Gbps. Despite targeting high-performance scenarios, they do not consider dynamic scenarios such as VANETs.

Saadallah Kassir et al. [10], studied a network architecture where vehicles form relay network clusters using Vehicle to Vehicle (V2V) and Vehicle to Infrastructure (V2I) links to improve connectivity and throughput on highways. The results highlight that multi-lane highways and vehicle control over relative positions further enhance performance, optimizing the connectivity-throughput trade-off.

The current state-of-the-art lacks implementations of P4 and IPv6 in VANET, which this work is expected to help address.

III. P4-NEON

This section details the P4-NEON using P4 code capable of handling SRv6 and INT operations in a single packet.

A. Architecture

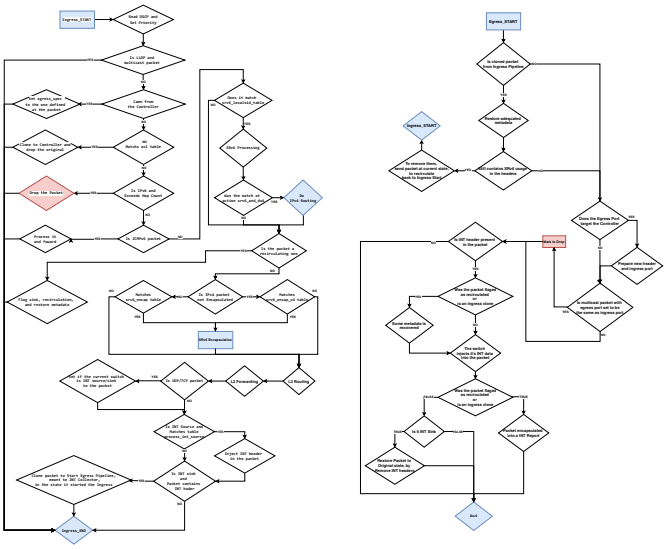
The data plane consists of P4 switches, which can be directly connected to hosts, route traffic, and are capable of performing SRv6 and INT operations. In the control plane, there is an entity that receives the INT reports and stores those data in a database, which many modules access instantaneously, Grafana generates a dashboard to display statistics, the Topology Visualizer illustrates the routes taken by the flows within the topology, and the INT The analyzer detects overloaded switches and attempts to reroute data flows by pushing SRv6 rules via the SDN controller.

B. P4 Switch

The topology includes P4 switches, with the Stratum Behavioral Model (BMv2) framework, which implements the V1Model architecture. Switches support the forwarding of packets meant to multicast and broadcast, but also packets: Neighbor Discovery Protocol (NDP); SRv6 (which are IPv6 packets); and packets Internet Protocol version 4 (IPv4) that were encapsulated into IPv6 when SRv6 operations are targeted at them. The logic applied to a packet as it arrives at the switch is represented in Fig. 1a. The logic applied to a packet as it leaves the switch is represented in Fig. 1b.

C. Telemetry

The INT data is stored in a time series database with four tables, all of which are filled by the INT Collector, as summarized in Table I. Each table represents a measurement.



(a) P4 Pipeline Ingress (b) P4 Pipeline Egress

Fig. 1: P4 Pipeline Ingress and Egress

Flow Stats contains general information about the packet. *Switch Stats* includes information on the processing performed by each switch where the packet was processed.

A switch identifies itself as the INT source by matching the packet's Internet Protocol (IP)s and flow label with an internal table, which is pre-configured by the SDN controller. If a match is found, the switch injects a INT header into the packet, and the original headers suffer some alterations. Still, the original information is backed up and later on restored.

The data is generated using the mode In-band Network Telemetry Embed Data (INT-MD), which means that in addition to the source switch injecting the instruction header (with a length of 12 bytes), each transit switch, through which the packet travels, will also inject a 4 bytes header with statistical data, causing the packet size to grow in Bytes as per Eq 1:

$$pktSize = 12 + 4n,\ n\ - Number\ of\ Transit\ Nodes \quad (1)$$

Measurement	flow_stats	switch_stats
Tags	`src_ip; dst_ip; flow_label dscp`	`switch_id; src_ip; dst_ip; flow_label dscp`
Time	`metric_timestamp`	`metric_timestamp`
Fields	`src_port; dst_port; protocol; size; latency; path`	`latency; size`

TABLE I: Flow Stats (entry per packet) and Switch Stats (entry per switch's packet processing)

All switches are set as transit switches, and in the topology, at most one packet can pass through 5 switches, so in the worst scenario, a packet can grow to 32 $Bytes$. This can eventually exceed the network's Maximum Transmission Unit (MTU), which is a well-known problem of INT, and has already been investigated with possible solutions [11]. We created our tests in such a way that such a problem would not occur.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:43:26 UTC from IEEE Xplore. Restrictions apply.

D. INT Collector

The INT collector runs a script that receives the INT reports, processes, and stores their information, but also other values such as the original packet's IPs, flow label, and Differentiated Service Code Point (DSCP). The collector is inserted into the network as a host, ideally close to SDN Controller.

E. INT Analyzer

The INT Analyzer implements the decision algorithm, which is described in Algorithm 1. This component is responsible for reading the generated telemetry in the Database, analyzing it to get the updated state of the topology and the load on the diverse switches, and, if necessary, creating/removing flow detours in the topology, via the usage of the Open Network Operating System (ONOS)'s Command-line Interface (CLI) to use the SRv6 operations. It is programmed to run at $t_{freq} = 15 seconds$, alternating between the creation and removal of SRv6 rules. This timing is to ensure that modifications are performed in the topology.

Algorithm 1: Logic to Create/Remove SRv6 Rules

```
Input: Telemetry from Database, Polling Interval T_s = 15s
Output: Alternating SRv6 Rule Creation/Removal
Initialize Operation as FALSE
Initialize Overloaded_Switches as an empty list
Initialize Created_Rules as an empty map
while True do
    for Each Switch do
        Calculate Load_Score using the INT from last T_s seconds
    end
    if Operation is FALSE then
        for Each Switch do
            if Load_Score ≥ 0.7 then
                Compute the load of flows passing at the switch
                  that do not have it as an end-point
                Sort the flows in descending order of load
                for Each Flow in the List do
                    if Not detoured in last T_s then
                        Request Controller to detour it
                        if Detour worked then
                            Save info in Created_Rules
                            break
                        end
                    end
                end
            end
        end
    else
        for Switches in Overloaded_Switches do
            if Load_Score ≤ 0.6 then
                Disable/Delete its SRv6 rules via Controller
                Remove switch from Overloaded_Switches
            end
        end
    end
    Set Operation to Not Operation
    sleep(T_s)
end
```

The component determines the current load of each switch via MCDA, and the resulting output is a single value between 0 and 1. P4-NEON considers 4 normalized values with their weights, as illustrated in Table II. We utilized the variables obtained through INT that we deemed most relevant and converted them to averages so that we could convert the entire time window into only 1 value. Consequently, MCDA was employed as a straightforward method of incorporating multiple variables into decision-making. The MCDA can be implemented using libraries like Numpy [12].

Variables	Min	Max	Weight
N. packets at switch	0	The total packets transmitted on current time window	0.1
AVG packet size	0	MTU	0.1
AVG packet processing time	0	85 percentile of processing time (current time window, all switches)	0.5
Not-infra switch	0	1	0.3

TABLE II: MCDA Variables

P4-NEON seeks to enhance processing efficiency and minimize flow latency, thus assigning a high weight of 0.5 to "Packet Processing Time". Given that non-infrastructure switches are anticipated to handle less traffic than infrastructure switches, whether the switch is a vehicle (non-infrastructure) becomes the second crucial factor; these are more likely to be marked as overloaded compared to infrastructure switches. Therefore, "Not-infrastructure switch" carries a weight of 0.3. The remaining two metrics are regarded as equally important. Unfortunately, queue-related data was not used due to it being unavailable since BMv2 does not support it natively. A switch is considered overloaded if its load is equal to or greater than 70%, and if it drops below 60%, it is no longer classified as overloaded.

F. Monitoring and Visualization

P4-NEON supports monitoring through a topology visualizer component, which allows one to see the actual paths used in the topology, and through Grafana dashboards.

IV. Evaluation Methodology

The evaluation employed Mininet and the network is managed by a ONOS controller, the database system used was InfluxDB, and the Visualizer scripts, INT Analyzer, and INT Collector were implemented via Python. No "extern" functionalities were used on this project's P4 code, as such it is non-dependent on hardware-specific features, making it easy to implement in real switch hardware, especially if this one also uses the V1model architecture. P4-NEON is available as an open-source project [13]. Fig. 2 is a visual representation of the testbed evaluating P4-NEON.

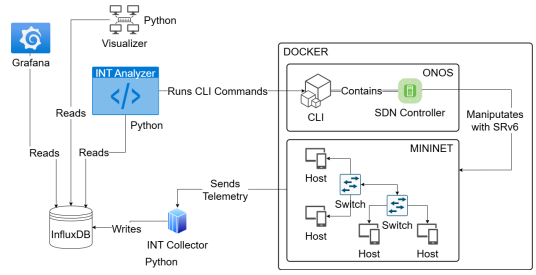


Fig. 2: Implemented Testbed

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:43:26 UTC from IEEE Xplore. Restrictions apply.

A. INT Collector

Since INT does not report a packet's original size, and we trim our report's body over privacy concerns, we cannot just undo the INT operations and obtain the original size. To overcome this limitation, the data flows on P4-NEON have a DSCP value, which is associated with the respective packet size; this way the INT collector just needs to read the DSCP value and match it with a JavaScript Object Notation (JSON) file that associates the DSCP values with the packet sizes. The associations used can be seen in Table III.

Represents	Sources	DSCP	Size
Message chat	[14]	0	262
Audio streams Spotify (Very high 320kbps)	[15]	34	420
Video Streams - YouTube 480p	[16]	35	874
Emergency VoIP Stream	[17]	46	100

TABLE III: DSCP values and corresponding size in bytes

B. Topology

Fig. 3 is a visual representation of the topology used in the tests, as perceived by ONOS and configured with Mininet.

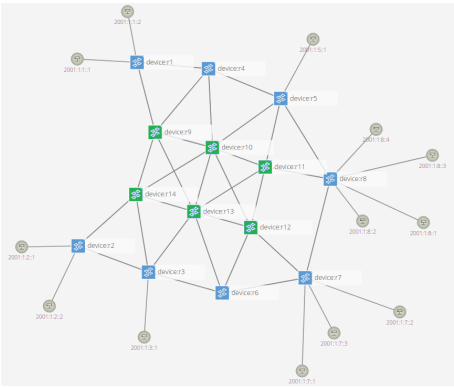


Fig. 3: Overall Topology

The topology was built to simulate a VANET. Thus, the blue elements, r1-r8 switches, are vehicles connected to the network through their OBU, and may contain hosts connected to them that generate/receive data flows. The elements in green, r9-r14 switches, are the RSU and represent the infrastructure in the area where vehicles circulate and are connected to them wirelessly. The 13 gray elements represent the hosts.

To streamline the collection of INT Telemetry reports during the simulation, the INT sink switches were configured with a port directly linked to the INT collector host.

Although Mininet is unable to independently simulate wireless environments, it does not significantly impact our goal of testing and validating the load balancing algorithm. Therefore, we established four distinct link types within the topology, each with its own configuration to closely mimic a real-world context for infrastructure, vehicles, and host connections, as outlined in Table IV. The Hierarchical fair-service curve (hfsc) scheduler is employed for all interfaces.

An **Infrastructure to Infrastructure (I2I)** connection is meant to represent a high-speed wired connection in the city infrastructure, such as optical fiber cables. We assume a cable of $150meters$ to calculate some metrics such as link delay, also a real Optical Multimode (OM)4 fiber cable is capable of $10Gbit/s$, but was configured with the hfsc scheduler for $1Gbit/s$. The **V2I** connection represents the wireless connection between the city's infrastructure and vehicles, so we assume it as a 5G long-distance connection since it is optimized for low latency and Internet of Things (IoT). The **V2V** connection represents the wireless connection between the vehicles, as such, we also assume a 5G connection but for short distances, so we assume a slightly lower delay and jitter due to the distance difference. The last **Vehicle to Host (V2H)** is the link between the vehicle and its hosts, which are hosts within its structure. These links are simulated as Wi-Fi 6 connections with a channel bandwidth of $40MHz$.

All interfaces are configured with a maximum queue size of 1000 packets except V2I, since the vehicles in the simulation are meant to be treated as having fewer resources than an RSU.

Properties	I2I	V2I	V2V	V2H
Connection	OM4 Optical Cable	5G	5G	Wi-Fi6
Sources	[18] [19]	[20]	[20]	[21]
Delay (ms)	30.6	13	10	30
Jitter (ms)	5	2.725	2	0.179
Loss (%)	0.1	0.1	0.1	0.5
Bandwidth (Mbit/s)	1000	64	64	88
Max Packets Queue	1000	1000	800	1000

TABLE IV: Link Types

C. Routing Methods

Three routing methods were used in each test scenario: 1- K Shortest path routing (KShort), 2- ECMP, and 3- ECMP+SRv6. **KShort** acts as the baseline for performance, all flows with a certain destination switch are sent to the same next hop. **ECMP** is similar to KShort, but with basic load balancing support, if there is more than 1 possible path with optimal cost, the several data flows are distributed. ECMP uses a hash (2 IPs and flow label) and converts the result into an integer between $[0, nPaths - 1]$, which is used to select and assign the path for each data flow, this approach has the risk of hash collisions occurring in ONOS during the assignment of paths, making multiple data flows take the same path, which does not take into account the availability of other unused or less congested paths. The **ECMP+SRv6** serves as the execution method for P4-NEON, distinguishing itself from the traditional ECMP approach by involving the INT Analyzer. This component evaluates when to use alternative routes for the data flows, thereby enhancing the load distribution of ECMP, via the SRv6 rules, to create path detours through less used topology nodes, and to remove said rules when they are no longer needed.

D. Evaluation Scenarios

To evaluate P4-NEON we considered four types of applications: Messaging, Audio streaming, Video Streaming, and Emergency Voice over Internet Protocol (VoIP) application. The characteristics are illustrated in Table V. The traffic of

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:43:26 UTC from IEEE Xplore. Restrictions apply.

such applications relied mainly on unidirectional UDP data flows and was simulated between topology hosts. The IP precedence of a packet is defined through the three most relevant bits of the respective DSCP value, it is used as the forwarding priority of a packet, where 0 is the default priority, 1 is the lowest, and 7 is the maximum [22].

Application	Sources	Interval (s)	Size (bytes)	DSCP	IP Precedence
Message chats	[14]	0.1	262	0	0
Audio streams	[15]	0.01	420	34	4
Video streams	[16]	0.020	1250	35	4
Emergency VoIP	[17]	0.020	100	46	5

TABLE V: Applications and flow characteristics

Different scenarios were defined with the flow of the applications, so different loads could be introduced. Three types of load were considered: 1- medium, 2- high, and 3- high + emergency. The **Medium Load** is a basic scenario with seven data flows, which, when combined, are not very intense.

- 2 Message chats: $(h8_1 \rightarrow h1_1)$ $(h2_1 \rightarrow h3_1)$
- 2 Audio streams: $(h1_2 \rightarrow h7_1)$ $(h5_1 \rightarrow h7_2)$
- 3 Video streams: $(h2_2 \rightarrow h8_2)$ $(h3_1 \rightarrow h8_3)$ $(h3_1 \rightarrow h7_3)$

The **High Load** scenario with 12 data flows, adding to the previous scenario the following data flows, to further assess the performance of the routing methods.

- 2 Audio streams: $(h3_1 \rightarrow h5_1)$ $(h8_1 \rightarrow h2_1)$
- 3 Video streams: $(h2_1 \rightarrow h8_1)$ $(h7_3 \rightarrow h8_4)$ $(h5_1 \rightarrow h2_2)$

The **High+Emergency Load** scenario is similar to the High load scenario, but with an additional Emergency VoIP data flow. This scenario is intended to see how the routing methods improve the performance of the emergency data flows that are already prioritized by the P4 switches. The new data flow added to the High Load scenario, for a total of 13 data flows:

- 1 Emergency VoIP Stream: $(h8_4 \rightarrow h1_2)$

V. Evaluation Results

For each combination of scenario and routing method, 10 runs were executed, and a percentile of 95% was used on the results, and we compare the performance of the P4-NEON proposal with the KShort and ECMP approaches.

The focus of P4-NEON was to improve ECMP performance by using SRv6. The performance of P4-NEON is assessed primarily based on the variation that occurs from the routing method ECMP to ECMP+SRv6, as P4-NEON uses MCDA for an efficient load balancing approach. The results are discussed considering the metrics AVG, STD, or Cumulative Distribution Function (CDF). As these allowed to highlight the main differences of the SRv6 algorithm.

Table VI displays a comparison of SRv6 operations executed in each test scenario. Notably, the INT Analyzer not only sets up SRv6 rules in some iterations but also removes them post-creation, demonstrating our component's ability to discern when a created rule becomes redundant for load balancing. By undoing unnecessary rules, the system enhances network performance, showcasing the component's capability to adapt dynamically to changes in the network environment.

Figures 4a, 4b, and 4c, clearly illustrate that incorporating SRv6 load balancing results in a decreased count of **out-of-order packets** within the network, shown by its consistently lower line compared to that of ECMP. Figures 4d, 4e, and 4f. illustrates that **flow latency** generally improves in the SRv6 scenario under most conditions, although towards the end, the highest latency values for SRv6 are higher than ECMP.

Referring to Table VII, the **packet distribution within the network** shows little change in AVG values, but STD values have notably improved by up to -30.33%, reducing the gap between the most and least loaded switches, suggesting enhanced load balancing.

In Table VIII, in the HIGH+EMERGENCY scenario, the emergency data flows are generally slower than the AVG of other flows. However, this gap narrows significantly with a more advanced routing algorithm. Our approach, using ECMP+SRv6, improves latency variation by 83.98% compared to standard ECMP.

VI. Conclusion

In fact, INT analyzer, by applying the creation or removal of SRv6 rules, can introduce improvements in the network by reducing the number of out-of-order packets, and the difference between the more and less loaded switches, and as a result, it also improves the performance of emergency flows compared to the others, without jeopardizing other measured metrics in our tests. For such, the metrics collected and used by MCDA constitute a feasible solution to be deployed in SDN controllers to enhance the load balancing and flow latency of high-priority data flows.

Our approach, which uses SRv6, is more efficient to detour traffic than just update the routing tables of certain switches, because it only needs to send one update to the source switch, so that it starts/stops changing all the packets of the targeted data flow, by adding to them the Segment Routing (SR) instructions to be applied, and the other switches, being SRv6-enabled, will know what to do, resulting in a faster, more reliable, and data-efficient process.

P4-NEON addresses the challenge of applying decision-making by using an efficient, easy-to-implement, training-free load-balancing algorithm. It also presents a real-time network performance assessment methodology supported by integrated dashboards, all implemented in P4 code that applies INT and SRv6 operations simultaneously or independently, while extending the focus to IPv6.

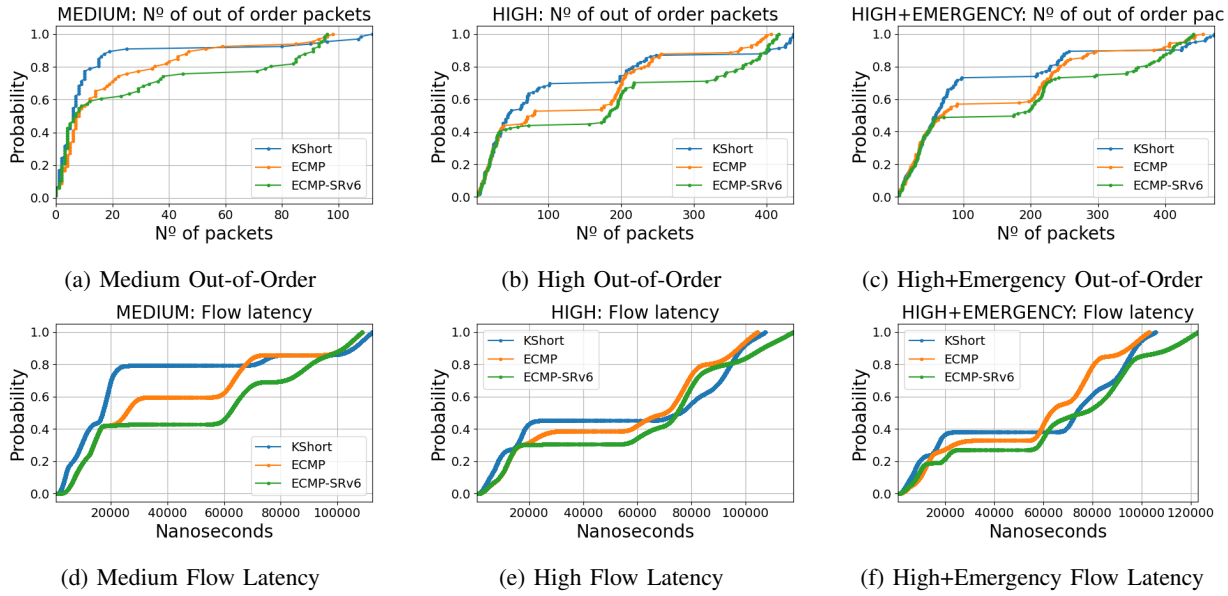
Acknowledgment

This work was supported by MH-SDVanet: Multihomed Software Defined Vehicular Networks (reference PTDC/EEI-COM/5284/2020). This work is also funded by national funds

Operation/Scenario	MEDIUM	HIGH	HIGH+EMERGENCY
AVG rules Created	1	2	3.9
AVG rules Removed	0	0	0.4

TABLE VI: SRv6 Operations done in each Test Scenario

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:43:26 UTC from IEEE Xplore. Restrictions apply.



(a) Medium Out-of-Order (b) High Out-of-Order (c) High+Emergency Out-of-Order

(d) Medium Flow Latency (e) High Flow Latency (f) High+Emergency Flow Latency

Fig. 4: Comparison of algorithms for Out-of-Order Packets and Flow Latency across different network scenarios.

	MEDIUM	HIGH	HIGH+EMERGENCY
AVG	27.79 → 27.77 (-0.7%)	28.03 → 28 (-0.11%)	27.93 → 27.9 (-0.11%)
STD	20.01 → 13.94 (-30.33%)	18.91 → 15.07 (-20.31%)	18.83 → 13.9 (-26.18%)

TABLE VII: Variation of % Packets per Switch from ECMP to ECMP+SRv6

Algorithms	KShort	ECMP	ECMP + SRv6
Values	46.78%	23.35%	3.74%
Change	None	46.78 → 23.35 (-50.09%)	23.35 → 3.74 (-83.98%)

TABLE VIII: Variation of the AVG Flow Latency from a non-Emergency flow to an Emergency one in the scenario HIGH+EMERGENCY

through the FCT - Fundação para a Ciência e a Tecnologia, I.P., in the framework of the Project UIDB/00326/2025 and UIDP/00326/2025.

References

[1] A. C. Michael Green, Anthony Thomas, "Fhwa white paper on mobile ad hoc networks," https://highways.dot.gov/research/publications/ear/FHWA-HRT-18-027, January 2018.

[2] S. Ibanez, G. Brebner, N. McKeown, and N. Zilberman, "The p4 netfpga workflow for line-rate packet processing," in *Proceedings of the 2019 ACM/SIGDA International Symposium on FGPGA*, 2019, p. 1–9.

[3] N. Ferreira, M. Luís, P. Rito, R. Rosmaninho, and S. Sargento, "Multi-homing in software defined vehicular networks," in *2024 IEEE Vehicular Networking Conference (VNC)*, 2024, pp. 132–139.

[4] "In-band network telemetry (int) dataplane specification," https://p4.org/p4-spec/docs/INT_v2_1.pdf, November 2020, accessed on 23.04.2025.

[5] Singh Harminder et al., "Int-sdn: Evaluation of various p4 parameters using optical telemetry having reconfigurable data plane on 40 gbps line rate," in *2019 IEEE PACRIM*, 2019, pp. 1–6.

[6] A. Gulenko, M. Wallschläger, and O. Kao, "A practical implementation of in-band network telemetry in open vswitch," in *2018 IEEE 7th International CloudNet*, 2018, pp. 1–4.

[7] L. C. de Almeida, R. Pasquini, and F. L. Verdi, "Using machine learning and in-band network telemetry for service metrics estimation," in *2021 IEEE 10th CloudNet*, 2021, pp. 33–39.

[8] netgroup, "p4-srv6," https://github.com/netgroup/p4-srv6, 2021, accessed on 1.12.2023.

[9] R. Kuntz, J. Montavont, and T. Noel, "Multihoming in ipv6 mobile networks: progress, challenges, and solutions," *IEEE Communications Magazine*, vol. 51, no. 1, pp. 128–135, 2013.

[10] S. Kassir, P. Caballero, G. de Veciana, N. Wang, X. Wang, and P. Palacharla, "Performance analysis of rsu-based multihomed multilane vehicular networks," 2020.

[11] Q. Yuan, F. Li, T. Pan, Y. Lai, Y. Gu, and X. Wang, "Int-segment: Mtu-adaptive single-path in-band network-wide telemetry," in *2022 IEEE 30th ICNP*, 2022, pp. 1–11.

[12] Charles R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020.

[13] D. C. Caetano, "p4-srv6-int," https://github.com/davidcc73/p4-srv6-INT, 2024, accessed on 06.08.2024.

[14] PlanHub.ca, "Whatsapp data usage: Understanding your messaging app's impact," https://www.planhub.ca/blog/en/whatsapp-data-usage/, March 2024, accessed on 23.04.2025.

[15] R. F. Mitja Rutnik, "How much data does spotify use? probably less than you think," https://www.androidauthority.com/spotify-data-usage-918265/, February 2024, accessed on 23.04.2025.

[16] ElectronicsHub, "How much data does youtube use?" https://www.electronicshub.org/how-much-data-does-youtube-use/, September 2024, accessed on 23.04.2025.

[17] W. Engineers, "Voip bandwidth requirements," https://www.erlang.com/support/voip-bandwidth/, accessed on 23.04.2025.

[18] K. Miller, "Calculating optical fiber latency," https://www.m2optics.com/blog/bid/70587/calculating-optical-fiber-latency, January 2012, accessed on 22.04.2025.

[19] Vincent, "Om4 multimode fiber faq: High-speed connectivity guide," https://www.fs.com/blog/om4-multimode-fiber-faq-highspeed-connectivity-guide-9498.html, December 2023, accessed on 22.04.2025.

[20] T. Daengsi, P. Ungkap, and P. Wuttidittachotti, "A study of 5g network performance: A pilot field trial at the main skytrain stations in bangkok," in *2021 ICAICST*, 2021, pp. 191–195.

[21] L. K. Omri Barkay, David Birnbaum, "IT@Intel: Wi-Fi 6–The Advantages Are Real," November 2021.

[22] J. Babiarz, K. Chan, and F. Baker, "Configuration guidelines for diffserv service classes," RFC Editor, RFC 4594, Aug. 2006.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on January 18,2026 at 16:43:26 UTC from IEEE Xplore. Restrictions apply.